
Lab Assignment # 10

Name of Student : B. Sai Rithvik
Enrollment No. : 2303A51536
Batch No. : 22

Task - 1(Error Detection and Correction)

Use AI to analyse a Python script and correct all syntax and logical errors.

Sample Input Code:

```
def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total
```

Expected Output-1:

Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

Prompt:

Generate a python program to fix the syntax error in the code and make it executable, and show the output of the code after fixing the syntax error.

Code:

```
# Task-1
# Generate a python program to fix the syntax error in the code and make it executable, and show the output of the code after fixing
def calculate_total(nums):
    # This function calculates the total sum of a list of numbers.
    # It takes a list of numbers as input and returns the total sum.
    # Note: Avoid using 'sum' as a variable name since it shadows the built-in function 'sum()'.
    total = 0
    for n in nums:
        total += n
    return total

# Test the function
if __name__ == "__main__":
    numbers = [49,58,59,66,67]
    result = calculate_total(numbers)
    print(f"Total sum: {result}")

#Justification:
# The original code had a syntax error due to the use of 'sum' as a variable name, which is a built-in function in Python.
# By renaming the variable to 'total', we avoid this conflict and ensure that the code is executable and produces the correct output
```

Output:

```
op/Python/AI_AC/Lab/Assignment-10.py
Total sum: 299
```

Justification:

The original code had a syntax error due to the use of 'sum' as a variable name, which is a built-in function in Python.

By renaming the variable to 'total', we avoid this conflict and ensure that the code is executable and produces the correct output. The function now correctly calculates the total sum of the list of numbers and prints it.

Task Description -2(Code Style Standardization)

Use AI to refactor Python code to comply with standard coding style guidelines.

Sample Input Code:

```
def findSum(a,b):return a+b  
print(findSum(5,10))
```

Expected Output-2:

Well-structured, consistently formatted Python code following standard style conventions.

Prompt:

Generate a python program to improve the code readability and maintainability by following best practices such as proper indentation, spacing, and naming conventions, and show the output of the code after improving its readability and maintainability.

Code:

```
# Task-2  
# Generate a python program to improve the code readability and maintainability by following best practices such as proper indentation  
def find_sum(a, b):  
    # This function finds the sum of two numbers.  
    # It takes two numbers as input and returns their sum.  
    return a + b  
  
print(find_sum(5, 10))  
# Justification:  
# The original code may have had issues with indentation, spacing, or naming conventions that made it less readable and maintainable.  
# By ensuring proper indentation, spacing, and using descriptive function and variable names, we enhance the readability and maintainability.
```

Output:

```
op/Python/AI_AC/Lab/Assignment-10.py  
15
```

Justification:

The original code may have had issues with indentation, spacing, or naming conventions that made it less readable and maintainable.

By ensuring proper indentation, spacing, and using descriptive function and variable names, we enhance the readability and maintainability of the code. This makes it easier for other developers to understand the purpose of the function and how to use it, as well as making it easier to identify and fix any potential issues in the future.

Task - 3(Code Clarity Improvement)

Use AI to improve code readability without changing its functionality.

Sample Input Code:

```
def f(x,y):  
    return x-y*2  
print(f(10,3))
```

Expected Output-3:

Python code rewritten with meaningful function and variable names, proper indentation, and improved clarity.

Prompt:

Generate a python program to implement a function that performs a specific task, and show the output of the code after implementing the function.

Code:

```
# Task-3  
#Generate a python program to implement a function that performs a specific task, and show the output of the code after  
def calculate_adjusted_difference(base_value, multiplier):  
    # This function calculates the adjusted difference between a base value and a multiple of it.  
    # It takes a base value and a multiplier as input and returns the adjusted difference.  
    # Note: The adjusted difference is calculated as base_value minus (multiplier times 2).  
  
    return base_value - multiplier * 2  
  
# Test the function  
if __name__ == "__main__":  
    result = calculate_adjusted_difference(10, 3)  
    print(f"Adjusted difference: {result}")
```

Output:

```
op/Python/AI_AC/Lab/Assignment-10.py  
Adjusted difference: 4
```

Justification:

The function 'calculate_adjusted_difference' is designed to perform a specific task of calculating the adjusted difference based on the provided base value and multiplier. By implementing this function, we can easily reuse it in different parts of the code whenever we need to calculate the adjusted difference, improving code modularity and maintainability. The output of the code will show the calculated adjusted difference based on the input values.

Task - 4(Structural Refactoring)

Use AI to refactor repetitive code into reusable functions.

Sample Input Code:

```
print("Hello Ram")  
print("Hello Sita")  
print("Hello Ravi")
```

Prompt:

Generate a python program to create a reusable function that can be called multiple times with different arguments, and show the output of the code after creating the reusable function.

Code:

```
# Task-4  
#Generate a python program to create a reusable function that can be called multiple times with different arguments,  
def print_greeting(name):  
    print(f"Hello {name}")  
  
print_greeting("Sai")  
print_greeting("Rithik")  
print_greeting("Venkataramana")  
# Justification:  
# The original code may have had repetitive code for printing greetings to different names. By creating a reusable fu
```

Output:

```
op/Python/AI_AC/Lab/Assignment-10.py  
Hello Sai  
Hello Rithik  
Hello Venkataramana
```

Justification:

The original code may have had repetitive code for printing greetings to different names. By creating a reusable function 'print_greeting', we can avoid code duplication and improve code organization. This modular design allows us to easily maintain and update the greeting logic in one place, rather than having to change it in multiple locations throughout the code. It also enhances readability and makes the code more efficient by reducing redundancy.

Task - 5(Efficiency Enhancement)

Use AI to optimize Python code for better performance.

Sample Input Code:

```
numbers = [ ]  
for i in range(1, 500000):  
    numbers.append(i * i)  
print(len(numbers))
```

Expected Output-5:

Optimized Python code that achieves the same result with improved performance.

Prompt:

Generate a python program to optimize the code for better performance by using efficient algorithms or data structures, and show the output of the code after optimizing it.

Code:

```
# Task-5  
# Generate a python program to optimize the code for better performance by using efficient algorithms or data structures, and show the output of the code after optimizing it.  
def generate_squares(n):  
    return (i * i for i in range(1, n))  
  
# Optimized approach using generator  
if __name__ == "__main__":  
    # Generator approach - memory efficient  
    squares_gen = generate_squares(500000)  
    count = sum(1 for _ in squares_gen)  
    print(f"Count of squares: {count}")  
    # Alternative: Using list comprehension (if storage needed)  
    # squares = [i * i for i in range(1, 500000)]  
    # print(len(squares))  
# Justification:  
# The original code may have used a less efficient approach to generate squares, such as using a list to store all the squares in memory.
```

Output:

```
op/Python/AI_AC/Lab/Assignment-10.py  
Count of squares: 499999  
PS C:\Users\bathi\OneDrive\Documents\Desktop\Python>
```

Justification:

The original code may have used a less efficient approach to generate squares, such as using a list to store all the squares in memory, which can lead to performance issues when dealing with large values of n. By using a generator expression, we can generate squares on-the-fly without storing them all in memory at once, which significantly improves performance and reduces memory usage. This approach allows us to handle larger inputs efficiently while still providing the necessary functionality.